

GUÍA DE REFERENCIA: COMANDOS ESENCIALES DE GIT

Manual práctico de configuración, control de versiones, reversión de cambios, ramas y trabajo colaborativo.

1. Comandos de Configuración

Permiten establecer la identidad del usuario y las preferencias globales o locales del entorno de Git.

Comando	Descripción y Efecto
<code>git config --global user.name "Nombre"</code>	Establece el nombre de usuario asociado a todas las confirmaciones (commits) que realices en el equipo.
<code>git config --global user.email "correo@ejemplo.com"</code>	Vincula tu dirección de correo electrónico a los commits firmados en tus repositorios.
<code>git config --global init.defaultBranch main</code>	Define 'main' como el nombre predeterminado para la rama principal al inicializar nuevos repositorios.
<code>git config --list</code>	Muestra todas las configuraciones actuales aplicadas en el entorno local y global de Git.

2. Comandos Principales (Control de Versiones Básico)

Utilizados para el flujo diario de trabajo local: inicializar proyectos, añadir cambios y registrar el historial.

Comando	Descripción y Efecto
<code>git init</code>	Inicializa un repositorio de Git vacío en el directorio actual, creando la estructura interna oculta (.git).
<code>git clone <url></code>	Clona un repositorio remoto existente en tu máquina local, descargando todo su historial y archivos.

<code>git status</code>	Muestra el estado actual del directorio de trabajo: archivos modificados, pendientes de confirmar (staged) y sin rastrear.
<code>git add <archivo></code>	Prepara (añade al área de preparación o staging area) los cambios de un archivo específico para el próximo commit.
<code>git add .</code>	Añade al área de preparación todos los archivos nuevos, modificados o eliminados del directorio actual.
<code>git commit -m "mensaje"</code>	Registra formalmente los cambios del área de preparación en el historial del repositorio con un mensaje descriptivo.
<code>git log</code>	Muestra el historial detallado de commits del repositorio con sus hashes, autores, fechas y mensajes.

3. Comandos para Revertir o Deshacer Cambios

Cruciales cuando necesitas dar marcha atrás a modificaciones no deseadas, corregir errores en commits recientes o limpiar el área de trabajo.

Comando	Descripción y Efecto
<code>git restore <archivo></code>	Descartar los cambios locales realizados en un archivo en el directorio de trabajo (vuelve al estado del último commit).
<code>git restore --staged <archivo></code>	Saca un archivo del área de preparación (staging area), revirtiendo el efecto de 'git add' sin perder los cambios del código.
<code>git reset --soft <commit></code>	Deshace los commits recientes pero mantiene intactos los cambios en el directorio de trabajo y el área de preparación.
<code>git reset --hard <commit></code>	Elimina por completo todos los commits y cambios realizados después del commit especificado, reseteando el código local de forma drástica.
<code>git revert <commit></code>	Crea un nuevo commit automático que invierte o deshace los cambios introducidos por un commit anterior, manteniendo limpio el historial compartido.

4. Comandos para el Trabajo con Ramas (Branching)

Esenciales para el desarrollo paralelo de características, corrección de errores y experimentación sin alterar la rama principal.

Comando	Descripción y Efecto
<code>git branch</code>	Lista todas las ramas locales existentes en el repositorio y resalta con un asterisco (*) la rama activa.
<code>git branch <nombre></code>	Crea una nueva rama local a partir del commit o rama actual en la que te encuentras.
<code>git switch <nombre></code> (o <code>git checkout <nombre></code>)	Cambia de la rama actual a la rama especificada, actualizando los archivos del directorio de trabajo.
<code>git switch -c <nombre></code> (o <code>git checkout -b <nombre></code>)	Crea una nueva rama y se cambia a ella de manera inmediata en un solo paso.
<code>git merge <rama></code>	Fusiona los cambios de la rama especificada en la rama actual en la que estás posicionado.
<code>git branch -d <nombre></code>	Elimina de forma segura una rama local que ya ha sido fusionada en el proyecto.

5. Comandos para el Trabajo Colaborativo

Permiten la sincronización y el intercambio de código entre repositorios locales y remotos (como GitHub, GitLab o Bitbucket).

Comando	Descripción y Efecto
<code>git remote add origin <url></code>	Vincula el repositorio local con un repositorio remoto especificado mediante su URL.
<code>git remote -v</code>	Muestra las URLs configuradas asociadas al repositorio remoto para operaciones de lectura y escritura.
<code>git fetch</code>	Descarga todos los cambios y ramas recientes del repositorio remoto sin fusionarlos automáticamente en tu código local.
<code>git pull</code>	Descarga los cambios del repositorio remoto y los fusiona de inmediato en tu rama local actual (equivalente a fetch + merge).

```
git push origin <rama>
```

Sube los commits de tu rama local especificada hacia el repositorio remoto correspondiente.